United International University
Department of Computer Science and Engineering

KROSH

Checkers Against Classical Search

Final Project Report

Submitted in partial fulfilment of the requirements of
CSE 3811 — Artificial Intelligence Laboratory
Section A · Group 10

Submitted by

Adham Zarif	0112330721
Saber Hassan	0112330870
Sadia Islam	0112330329

Submitted to

Kazi Fahim Tahmid
Lecturer, Department of Computer Science and Engineering
United International University

Date of Submission: 3rd October 2026

Abstract

KROSH is a full English-draughts (American checkers) system containing several classical adversarial-search engines arranged by strength: Greedy, Minimax, Alpha-Beta with a transposition table, killer moves, quiescence search, and iterative deepening, plus Monte Carlo Tree Search (MCTS). Its Hybrid engine changes methods by game phase, using Alpha-Beta during the opening and mid-game before handing play to MCTS in the end-game. Everything is written in pure Python, with a 32-slot The system combines a rules core with an engine-independent Flask API and a browser client written by hand. It supports both Human-vs-Computer and Human-vs-Human play. In the authors' benchmarks, Minimax was tested at depths 2–4, while Alpha-Beta was tested at depth 4, along with MCTS. 500 simulations and the Hybrid each won every game played against the Greedy baseline, and the Hybrid At depth 3, it beat Minimax 3–0. Minimax expanded 37 nodes per move at depth 2 and 1,420 at depth 4. The code base contains 4,982 lines: 3,520 in production code, 1,108 in tests, and 354 in benchmark scripts. Pytest reports 131 passing cases. The samples are small, so these results are indicative rather than statistically conclusive.

Keywords: adversarial search, minimax, alpha-beta pruning, Monte Carlo Tree Search, hybrid search, checkers.

Contents

1 Introduction **2**

1.1 Motivation 2

1.2 Baseline theory 2

1.3 Contributions 2

2 Related Work **3**

3 Methodology **3**

3.1 End-to-end pipeline 3

3.2 Board representation and move generation 4

3.3 Search engines 4

3.4 Hybrid engine 6

4 Implementation **7**

4.1 Code organisation 7

4.2 HTTP API and browser client 7

4.3 Testing and reproducibility 8

5 Results **8**

5.1 Minimax node growth 8

5.2 Head-to-head records 9

6 Discussion **9**

7 Future Work **9**

8 Conclusion **10**

1 Introduction

1.1 Motivation

Checkers has stood as a benchmark for artificial intelligence since Samuel's learning program in the 1950s. The game was settled in 2007, when the Chinook team proved that perfect play always leads to a draw [2]. That makes it a strong course-project choice: the rules take only a few hundred lines to implement, but the game still has plenty of depth. (branching factor roughly 4–8, with captures compulsory), making the differences among greedy, depth-limited, pruned, and simulation-based search clear in playing strength, running time, and node count.

The project asks *how much each classical search method improves play, and whether the added strength can be measured?. A simple phase-switching combination of them plays better than any single engine*

1.2 Baseline theory

Checkers is a two-player, zero-sum, perfect-information game. We use the standard game-tree formalism [3]: nodes are positions, edges are legal moves, and an engine picks the root move with the best backed-up value.

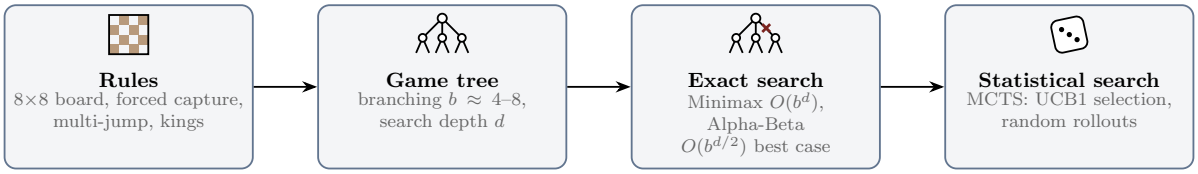


Figure 1: Theory background in one picture: from the rules to the search methods KROSH implements.

Minimax searches the tree to depth d in $O(b^d)$ time. Alpha-Beta prunes provably irrelevant subtrees and reaches $O(b^{d/2})$ with perfect move ordering [4]. MCTS instead estimates a move's value from many random playouts and balances exploration and exploitation with the UCB1 rule [5].

1.3 Contributions

- A complete 32-slot draughts core handles the full ruleset, including in-place apply/undo, compulsory multi-jump captures, and promotion.
- Five engines—Greedy, Minimax, Alpha-Beta, MCTS, and Hybrid—share one `Engine`/`SearchResult` interface.
- The configurable Hybrid delegates to different engines according to the number of pieces remaining, then shows that selection in the UI.
- A Flask API backs a plain HTML/CSS/JS client with live search telemetry.
- The project includes 131 Pytest cases, along with seeded benchmark scripts that produce reproducible results.

2 Related Work

Work	Idea	Relation to KROSH
Chinook [2]	Tuned evaluator, deep alpha-beta, opening book, endgame tablebases; solved checkers (2007).	KROSH reuses alpha-beta, but has no book, tablebase or tuned evaluator.
UCT / MCTS [5, 6, 7]	Bandit-based tree search with random rollouts; strong where branching is large (Go).	KROSH implements vanilla UCT with uniform rollouts.
AlphaZero [8]	MCTS guided by a learned policy/value network.	Listed as future work (learned prior).
Phase-switching hybrids	Use different search methods in different game phases.	KROSH's Hybrid follows this route with piece-count thresholds.

Table 1: Related work and how it connects to this project.

3 Methodology

3.1 End-to-end pipeline

Figure 2 presents the complete system. A browser click triggers an HTTP request, which the session checks against the rules core. When it is the computer's turn, the router forwards the position to the selected engine, which applies the rules core and shared static evaluator during its search, then returns a `SearchResult` for the UI to display.

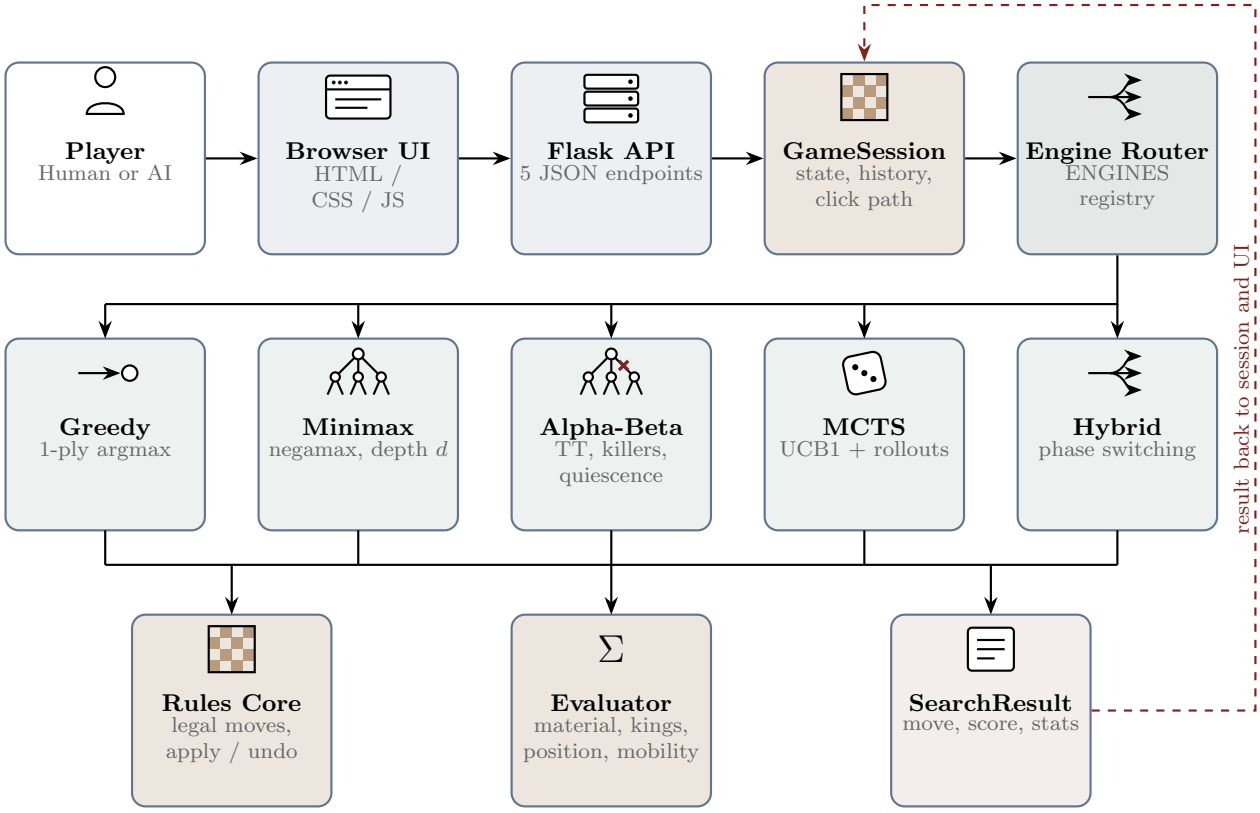
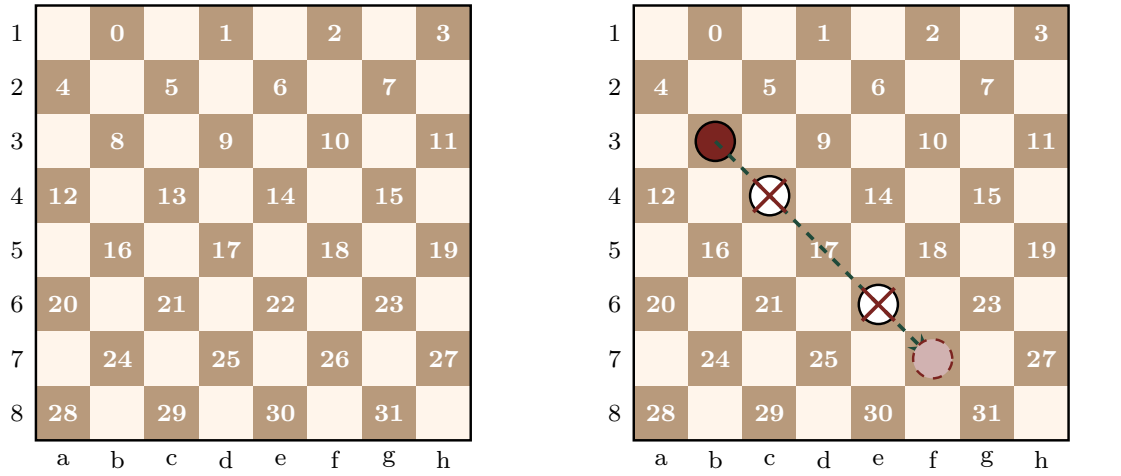


Figure 2: KROSH pipeline: browser → Flask API → session → engine router → one of five engines, all sharing the rules core and static evaluator and returning a uniform `SearchResult`.

3.2 Board representation and move generation

Only dark squares can hold pieces, which reduces the board to a flat array of 32 slots: 0 means empty, while positive and negative values distinguish men from kings. Move generation then becomes simple index arithmetic against an adjacency table. `legal_moves()` searches for captures first, recursively extending multi-jump sequences and recording each square taken along the way. Quiet moves are considered only when no capture is available, so the function returns one complete category or the other—never both. Every consumer automatically enforces mandatory capture. Searches perform in-place apply and undo actions using an undo stack instead of duplicating states.



(a) 32-slot numbering **(b) double jump: slots $8 \to 17 \to 26$**

Figure 3: (a) Slot numbering of the 32 dark squares. (b) A red man captures the white men on slots 13 and 22 in one compulsory multi-jump chain; the pipeline records the path and the captured slots in one `Move`.

3.3 Search engines

Greedy and Minimax. Greedy examines every legal move, evaluates the resulting position with the static evaluator, and chooses the highest score without looking beyond that move. Minimax is written in negamax form: score $= -\text{negamax}(\text{child}, d-1)$. That lets both players use the same code path.

Alpha-Beta. Alpha-Beta searches within the (α, β) window. Once a subtree can no longer affect the value at the root, the search stops there. Figure 4 shows the cutoff in a nine-leaf tree: plain minimax evaluates all 9 leaves, while Alpha-Beta evaluates 7. KROSH adds five enhancements (Figure 5) on top of the textbook algorithm.

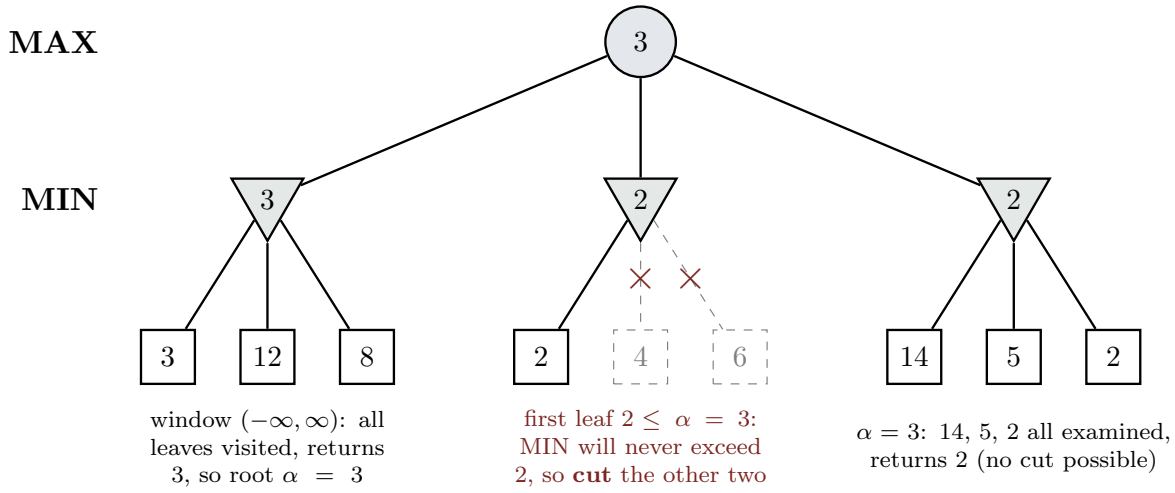


Figure 4: Worked alpha-beta trace. Dashed grey leaves are never visited; the root value (3) is identical to minimax.

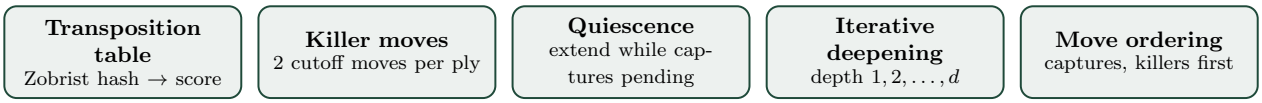


Figure 5: Enhancements layered on KROSH's Alpha-Beta engine (904 lines).

Monte Carlo Tree Search. MCTS cycles through four phases (Figure 6). During selection, it descends according to UCB1 $= \frac{wi}{ni} + c\sqrt{\ln N / n_i}$, where $c = \sqrt{2}$; any child that has not been visited receives infinite priority. Each rollout selects moves uniformly at random and stops after at most 20 plies. Rewards are negated when passed between layers, and the algorithm ultimately chooses the root child. with the most visits. Table 2 gives a numeric selection step (illustrative values).

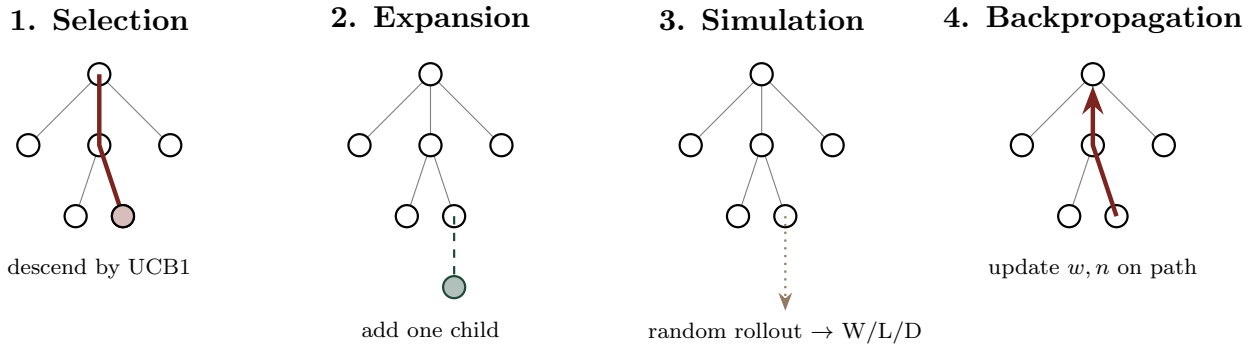


Figure 6: One MCTS iteration. KROSH runs 500 iterations in benchmarks and 5,000 in the Hybrid's endgame.

Child	w_i	n_i	w_i/n_i	UCB1 (N=10, c=$\sqrt{2}$)
A	3	5	0.60	1.56
B	2	3	0.67	1.91
C	1	2	0.50	**2.02** ← selected

Table 2: Illustrative UCB1 selection: the least-visited child wins despite a lower win rate (exploration term).

3.4 Hybrid engine

The Hybrid holds one instance of each inner engine. On every call, it counts the pieces, then sends the work to the appropriate engine (Figure 7). Delegation keeps those engines independently testable instead of tying them together through inheritance. It also lets the Hybrid label each result with `extra.phase` and `extra.delegate`; the UI displays both fields live. This design rests on three hypotheses that we did *not* test by ablation: (i) deep search is wasted In the quiet opening, while alpha-beta performs best through the mid-game, the static evaluator becomes least dependable in sparse endgames, where MCTS rollouts are cleaner. Every threshold and budget is supplied through the constructor.

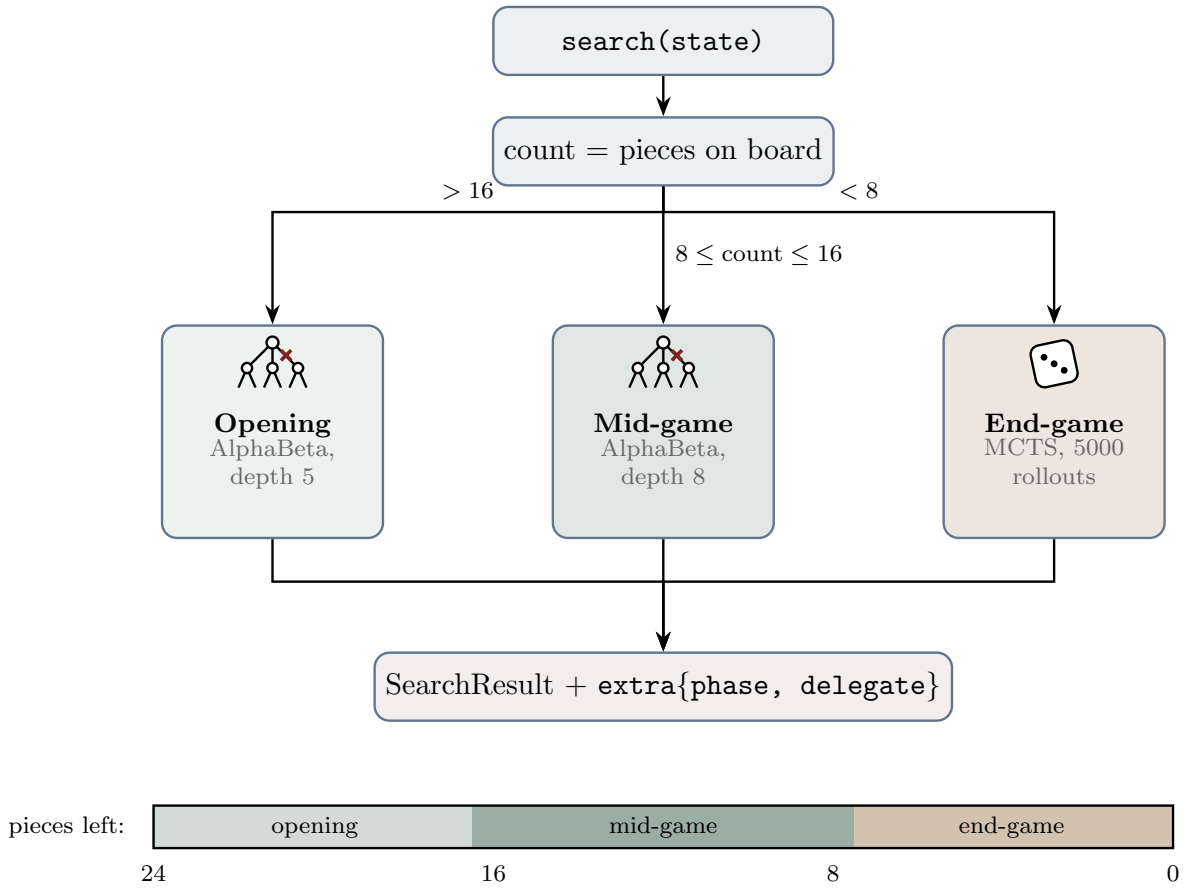


Figure 7: Hybrid phase switching. Bottom bar: which engine plays at each piece count (from 24 at the start to 0).

4 Implementation

4.1 Code organisation

Subsystem	Files	Lines	Notes
Rules core (`krosh/core/`)	4	449	`state.py` 305 lines
Search engines (`krosh/engines/`)	9	1,838	`alphabeta.py` 904, `mcts.py` 250, `hybrid.py` 111
Session layer (`krosh/app/`)	2	237	click path, history, undo
Web layer (`krosh/web/`)	5	996	Flask + HTML 118, CSS 428, JS 273
Production total	20	**3,520**	
Pytest suite (`tests/`)	5	1,108	131 test cases
Benchmark scripts (`scripts/`)	4	354	seeded runs
Everything	29	**4,982**	

Table 3: Code size by subsystem (`wc -l`, October 2026).

4.2 HTTP API and browser client

Endpoint	Method	Purpose
`/api/game`	POST	New session (engine, colour, player names)
`/api/game/<id>/click`	POST	Advance the current click path
`/api/game/<id>/undo`	POST	Undo the previous move
`/api/game/<id>/restart`	POST	Reset to the initial position
`/api/engines`	GET	List available engines

Table 4: Flask endpoints. The server is the single source of truth; the client never adjudicates a move.

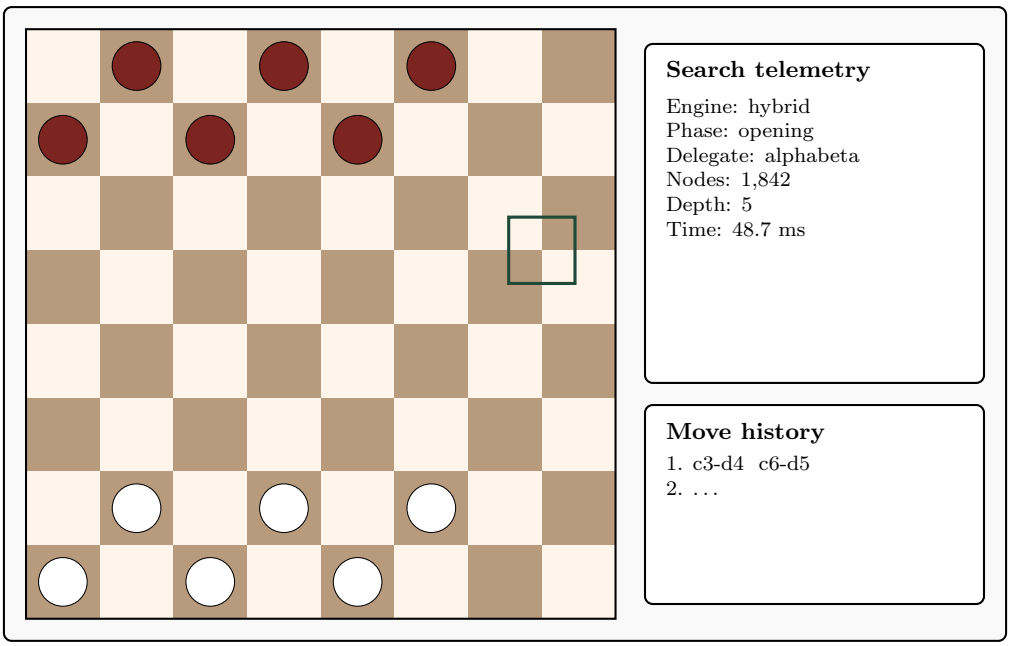


Figure 8: Schematic of the browser client: board with pulsing capturable pieces and soft target highlights (left), live telemetry and move history (right). Telemetry values are those of the API transcript in Appendix A.

Visual aids: capturable pieces show their pulse, making the forced-capture rule clear; destinations appear as translucent discs, kings wear an SVG crown, and the sidebar displays `extra.phase`/`extra.delegate`

from every response. Human-vs-Human mode uses the same rules core, with two custom player names.

4.3 Testing and reproducibility

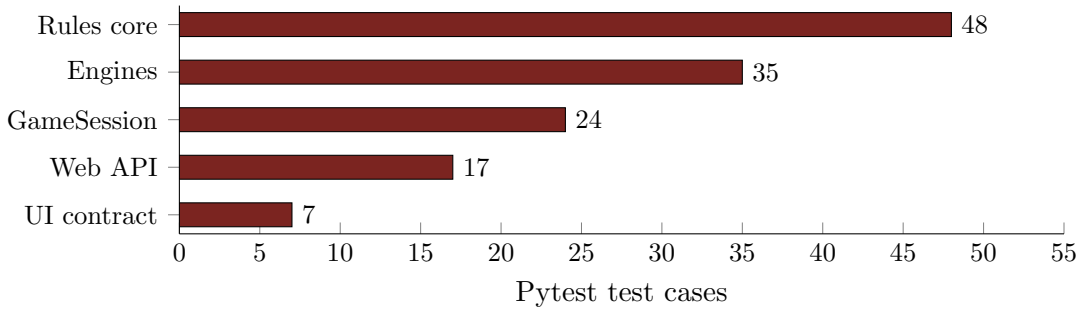


Figure 9: 131 Pytest test cases by subsystem; all pass.

The rules core is also tested from the starting position with a perft-style leaf count at depths 1–6. The reported totals match those published by the Open Checkers project. Each engine accepts an integer seed for tie-breaking and rollouts, making its moves reproducible when the same seed is used.

5 Results

All results were produced on the authors' machine running Windows 11 and Python 3.13.1 in October 2026. Each matchup consists of three games, with the first engine named playing red. Minimax is the exception: each depth uses six games, three with each colour.

5.1 Minimax node growth

Depth	W	L	D	Nodes/move	Time/move	Nodes growth	Time growth
2	6	0	0	37	0.8 ms	—	—
3	6	0	0	259	5.4 ms	7.0×	6.8×
4	6	0	0	1,420	31.8 ms	5.5×	5.9×

Table 5: Minimax vs. Greedy. W/L/D are Minimax results over six games per depth. The last two columns are per-ply growth.

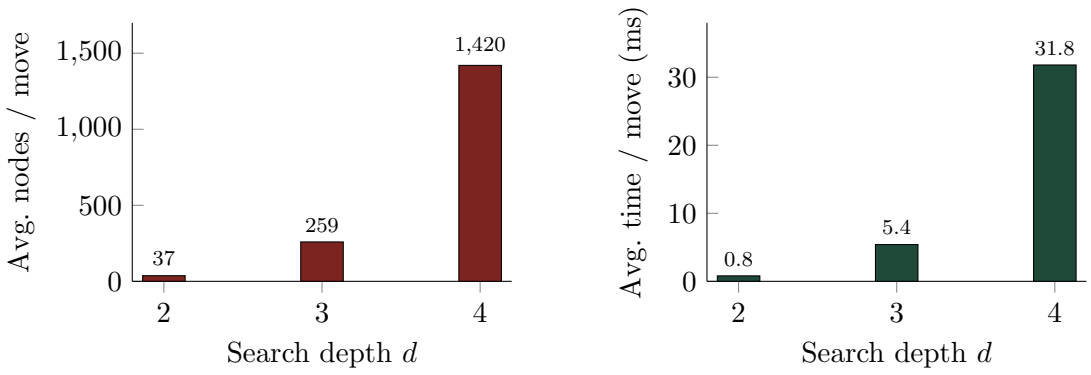


Figure 10: Minimax effort grows steeply with depth; time tracks node count.

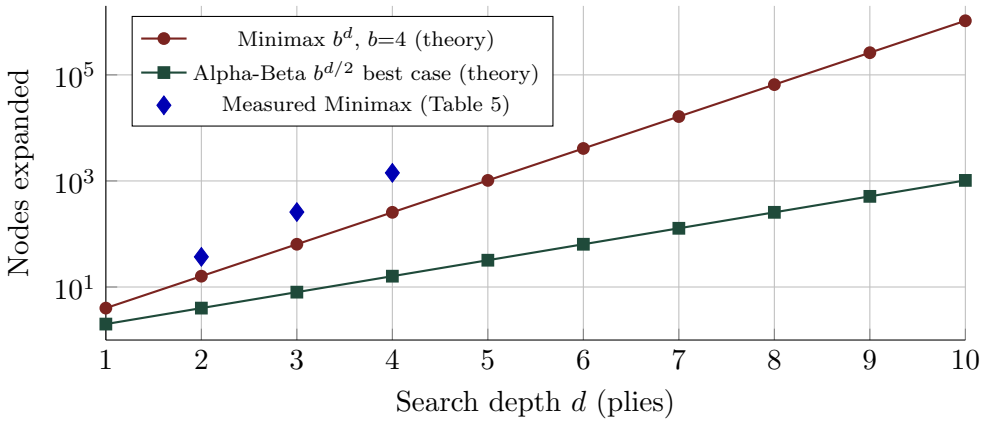


Figure 11: Measured Minimax nodes versus theory. Measured counts grow about 6× per ply, i.e. an effective branching factor near 6, higher than the assumed b=4. The Alpha-Beta curve is a theoretical best case, not a measurement.

5.2 Head-to-head records

Matchup (first-named as red)	W	L	D	Games	Note
Minimax (d=2, 3, 4) vs. Greedy	18	0	0	18	both colours
Alpha-Beta (d=4) vs. Greedy	3	0	0	3	red only
MCTS (500 sims) vs. Greedy	3	0	0	3	red only
Hybrid vs. Greedy	3	0	0	3	red only
Hybrid vs. Minimax (d=3)	3	0	0	3	red only

Table 6: Head-to-head results. No matchup pits the Hybrid against deep Alpha-Beta.

6 Discussion

What the data supports. Every advanced engine outperforms the one-ply Greedy baseline, while Minimax effort rises exponentially, as theory predicts. The Hybrid also surpasses depth-3 Minimax.

Limitations.

- **Small samples:** Each matchup covers only three games, usually with the Hybrid playing red. That leaves wide confidence intervals, and the Hybrid has not yet played Alpha-Beta.
- **No ablation:** We do not yet know whether the Hybrid wins because it switches phases or simply because it uses Alpha-Beta in most positions.
- **Hand-picked thresholds:** The 16/8 cutoffs and depths 5/8 were chosen from intuition rather than tuned against data.
- **Fixed budgets:** Engines allocate a set search depth or number of simulations instead of a time limit, which makes response times vary between phases.
- **Simple evaluator and rollouts:** focus on material, kings, position, and mobility; MCTS rollouts are uniform-random.

7 Future Work

1. Run a round-robin tournament in which the AI plays itself across many seeds and with both colours. Track Elo-style ratings, then sweep the threshold; this also supplies the missing ablation.
2. Expose time-limited iterative deepening through the API.

3. An opening book paired with a compact endgame tablebase.
4. An MCTS policy prior trained in the style of AlphaZero [8].
5. A browser-based dashboard displaying tournament results.

8 Conclusion

KROSH implements the classic game-theory AI ladder, featuring greedy search, minimax, and alpha-beta pruning with contemporary refinements, along with Monte Carlo Tree Search (MCTS). A configurable phase-switching Hybrid sits behind one interface, backed by a tested rules core and a playable web client. In our small benchmarks, every engine outperformed the Greedy baseline, while the Hybrid also beat depth-3 Minimax; Minimax Cost rose exponentially, just as predicted. A fair assessment of the Hybrid still requires the tournament and the ablation study listed above.

References

[1] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM J. Res. Dev.*, vol. 3, no. 3, pp. 210–229, 1959.

[2] J. Schaeffer *et al.*, "Checkers is solved," *Science*, vol. 317, no. 5844, pp. 1518–1522, 2007.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[4] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[5] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. ECML*, 2006, pp. 282–293.

[6] R. Coulom, "Efficient selectivity and backup operators in Monte-Carlo tree search," in *Computers and Games*, LNCS vol. 4630, Springer, 2007, pp. 72–83.

[7] C. B. Browne *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, 2012.

[8] D. Silver *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.